

Programming for AI

BS(AI) Part-2- 2nd Semester

Prof. Dr. Imtiaz Ali Korejo

Python Lists

Week- , Lecture-





Outline of Today's Lesson

1. Python List

1. Python List
2. Access List items
3. Add List Items
4. Remove List Items

2. Python Tuples

1. Python Tuples
2. Access Tuples
3. Update Tuples
4. Loop Tuples





Python Collections (**Arrays**)

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered(version 3.7 +) and changeable. No duplicate members.



Section-1

Python Programming

PYTHON LISTS



By: Prof. Dr. Imtiaz Ali Korejo





Python Lists

- Lists are used to store multiple items in a single variable.
- Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage.
- Lists are created using square brackets:

```
mylist = ["apple", "banana", "cherry"]
```





Python Lists

- List Example

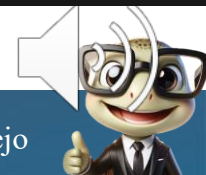
```
thislist = ["apple", "banana", "cherry"]  
print(thislist)
```

Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dr Imtiaz Korejo\Desktop\pythonworkspace>

['apple' , 'banana' , 'cherry']

Ln 9, Col 1 Spaces: 4 UTF-8 LF {} JavaScript Go Live





List Items

- List items are **ordered**, **changeable**, and **allow duplicate** values.
- List items are indexed, the first item has **index [0]**, the second item has **index [1]** etc.





Ordered

- Lists in python are ordered.
- It means that the items have a **defined order**, and that **order will not change**.
- If you add new items to a list, the new items will be placed at the **end of the list**.
- The order of elements in a list is determined by their **position (index)**, and when you add new items to the list, they are appended to the end, maintaining the order.





Python Lists

- List items can be of any data type:

```
list1 = ["apple", "banana", "cherry"]  
List2 = [1, 3, 5, 9, 3]  
List3 = [ True, False, False]  
print(thislist)
```

Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dr Imtiaz Korejo\Desktop\pythonworkspace>

```
[ 'apple' , 'banana' , 'cherry' ]  
[1, 5, 7, 9, 3 ]  
[True, False, False]
```





Python Lists

List Methods

Python has a set of built-in methods that you can use on lists.

Method	Explanation
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy ()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list



#Example 2: Inserting Items at Specific Positions

Create a list with some initial items

```
numbers = [10, 20, 30]
```

Print the list to show the initial order

```
print("Initial list:", numbers)
```

Insert a new item at a specific position (index 1)

```
numbers.insert(1, 15)
```

Print the list again to show the updated order

```
print("List after inserting 15 at index 1:", numbers)
```

Add another item to the end of the list

```
numbers.append(40)
```

Print the list to show the final order

```
print("List after appending 40:", numbers)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dr Imtiaz Korejo\Desktop\pythonworkspace>

Initial list: [10, 20, 30]

List after inserting 15 at index 1: [10, 15, 20, 30]

List after appending 40: [10, 15, 20, 30, 40]



Changeable

- The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.





Allow Duplicates

- Since lists are indexed, lists can have items with the same value:
- Lists allow duplicate values:

```
thislist = ["apple", "banana", "cherry", "apple", "cherry"]  
  
print(thislist)
```

Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dr Imtiaz Korejo\Desktop\pythonworkspace>

['apple', 'banana', 'cheery', 'apple', 'cherry']





List Length

- To determine how many items a list has, use the **len()** function:

The screenshot shows a code editor on the left and a terminal window on the right. The code editor contains the following Python code:

```
#Print the number of items in the list:  
thislist = ["apple", "banana", "cherry"]  
print(len(thislist))
```

The terminal window shows the output of the code:

```
Microsoft Windows [Version 10.0.22631.5624]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\Dr Imtiaz Korejo\Desktop\pythonworkspace>  
  
3
```





List Items - Data Types

- List items can be of any data type:
- String, int and boolean data types:

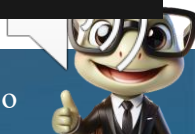
The screenshot shows a code editor on the left and a terminal window on the right. The code editor contains three lines of Python code: `list1 = ["apple", "banana", "cherry"]`, `list2 = [1, 5, 7, 9, 3]`, and `list3 = [True, False, False]`. The terminal window shows the output of these lists: `['apple', 'banana', 'cherry']`, `[1, 5, 7, 9, 3]`, and `[True, False, False]`.

```
list1 = ["apple", "banana", "cherry"]
list2 = [1, 5, 7, 9, 3]
list3 = [True, False, False]
```

```
Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dr Imtiaz Korejo\Desktop\pythonworkspace>

['apple', 'banana', 'cherry']
[1, 5, 7, 9, 3]
[True, False, False]
```





List Items - Data Types

- A list can contain different data types:
- A list may include strings, integers and boolean values, all in single list

```
list1 = ["abc", 34, True, 40, "male"]
```

The screenshot shows a code editor with a dark theme. The code defines a list named 'list1' containing five elements: a string 'abc', an integer 34, a boolean True, an integer 40, and a string 'male'. The editor has a search bar at the top, a file explorer on the left, and a status bar at the bottom showing 'Ln 9, Col 1', 'Spaces: 4', 'UTF-8', 'LF', and 'JavaScript'.

```
Microsoft Windows [Version 10.0.22631.5624]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\Dr Imtiaz Korejo\Desktop\pythonworkspace>  
  
['abc', 34, True, 40, 'male']
```

The screenshot shows a terminal window with a dark theme. It displays the output of the Python list definition: ['abc', 34, True, 40, 'male']. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, and PORTS. The status bar at the bottom shows 'Microsoft Windows [Version 10.0.22631.5624]' and '(c) Microsoft Corporation. All rights reserved.'





The list() Constructor

- It is also possible to use the `list()` constructor when creating a new list.
- Using the `list()` constructor to make a List:

```
# note the double round-brackets
thislist = list(("apple", "banana", "cherry"))

print(thislist)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
(.venv) zeeshanbhatti@Zeeshans-MacBook-Pro Python Lab Code %
['apple', 'banana', 'cherry']
```





type() Method in List

- From Python's perspective, lists are defined as objects with the data type 'list':

<class 'list'>

```
mylist = ["apple", "banana", "cherry"]
print(type(mylist))
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
<class 'list'>
```



Section-2

Python Programming

PYTHON – LIST OPERATIONS



By: Prof. Dr. Imtiaz Ali Korejo





Access Items

- List items are indexed and you can access them by referring to the **index** number:

The screenshot displays a code editor with a dark theme. The editor contains the following JavaScript code:

```
#Print the second item of the list:  
thislist = ["apple", "banana", "cherry"]  
print(thislist[1])
```

The code is executed, and the output 'banana' is shown in the terminal window on the right. The terminal window has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, PORTS, and COMMENTS. The status bar at the bottom of the editor shows 'Ln 9, Col 1', 'Spaces: 4', 'UTF-8', 'LF', and 'JavaScript'.





Range of Indexes

- You can specify a range of indexes by specifying where to start and where to end the range.
- When specifying a range, the return value will be a new list with the specified items.

List [startIndex : endIndex]

```
#Return the third, fourth, and fifth item:  
  
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon",  
            "mango"]  
  
print(thislist[2:5])
```

The terminal output shows the result of the slicing operation: ['cherry', 'orange', 'kiwi']





Default Start Index

- By leaving out the start value, the range will start at the first item:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[:4])
```

The terminal output shows the first four elements of the array: ['apple', 'banana', 'cherry', 'orange']





Default End Index

- By leaving out the end value, the range will go on to the end of the list:

•

```
1 thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon",  
"mango"]  
  
print(thislist[2:])
```

Terminal Output:

```
['cherry', 'orange', 'kiwi',  
'melon', 'mango']
```





Check if Item Exists

- To determine if a specified item is present in a list use the **in** keyword:

```
#Check if "apple" is present in the list:  
  
thislist = ["apple", "banana", "cherry"]  
  
if "apple" in thislist:  
    print("Yes, 'apple' is in the fruits list")
```

Yes, 'apple' is in the fruits list





Change Item Value

- To change the value of a specific item, refer to the index number:
-

```
#Change the second item:  
... thislist = ["apple", "banana", "cherry"]  
    thislist[1] = "blackcurrent"  
    print(thislist)
```

['apple', 'blackcurrent', 'cherry']

The screenshot shows a code editor with a dark theme. The code defines a list 'thislist' with three items: 'apple', 'banana', and 'cherry'. It then updates the item at index 1 to 'blackcurrent' and prints the list. The terminal on the right shows the output of the print statement: ['apple', 'blackcurrent', 'cherry']. The status bar at the bottom indicates the file is a JavaScript file, UTF-8 encoding, and line 9, column 1.





Change a Range of Item Values

- To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to insert the new values:
- Change the values "banana" and "cherry" with the values "blackcurrant" and "watermelon":

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]  
...  
thislist[1:3] = ["blackcurrant", "watermelon"]  
  
print(thislist)
```

['apple', 'blackcurrant', 'watermelon', 'orange', 'kiwi', 'mango']





Insert Items

- To insert a new list item, without replacing any of the existing values, we can use the **insert()** method.
- The **insert()** method inserts an item at the specified index:

```
thislist = ["apple", "banana", "cherry"]  
  
...  
thislist.insert(2, "watermelon")  
  
print(thislist)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS  
[ 'apple', 'banana', 'watermelon', 'cherry' ]
```





Append Items

- To add an item to the end of the list, use the **append()** method:

```
thislist = ["apple", "banana", "cherry"]  
  
thislist.append("orange")  
  
print(thislist)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS  
  
['apple', 'banana', 'cherry', 'orange']
```





Remove Specified Item

- The **remove()** method removes the specified item.

```
thislist = ["apple", "banana", "cherry"]  
...  
thislist.remove("banana")  
  
print(thislist)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
(.venv) zeeshanbhatti@Zeeshans-MacBook-Pro Python Lab Code %  
['apple', 'cherry']
```

Ln 9, Col 1 Spaces: 4 UTF-8 LF {} JavaScript Go Live





Remove Specified Index

- The **pop()** method removes the specified index.

```
thislist = ["apple", "banana", "cherry"]  
  
thislist.pop(1)  
  
print(thislist)
```

['apple', 'cherry']





Clear the List

- The **clear()** method empties the list.
- The list still remains, but it has no content.

```
1 thislist = ["apple", "banana", "cherry"]  
  
thislist.clear()  
  
print(thislist)
```

The screenshot shows a code editor with a dark theme. The code defines a list 'thislist' with three fruit names, then calls 'clear()' on it, and finally prints the list. The status bar at the bottom indicates 'Ln 9, Col 1', 'Spaces: 4', 'UTF-8', 'LF', and 'JavaScript'.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS  
[ ]
```

The screenshot shows a terminal window with a dark background. The 'TERMINAL' tab is selected. The output shows an empty list '[]'.





Loop Through a List

- You can loop through the list items by using a **for loop**:

```
thislist = ["apple", "banana", "cherry"]  
  
for x in thislist:  
    print(x)
```

apple
banana
cherry





Loop Through the Index Numbers

- You can also loop through the list items by referring to their index number.
- Use the `range()` and `len()` functions to create a suitable iterable.

```
thislist = ["apple", "banana", "cherry"]

for i in range(len(thislist)):
    print(thislist[i])
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

apple
banana
cherry
```



Section-4

Python Programming

REAL WORLD PROBLEMS



By: Prof. Dr. Imtiaz Ali Korejo



1. Accessing List Items: Filtering Data

Problem: You have a dataset of temperatures recorded over a week, and you need to access the temperatures above a certain threshold (e.g., 30°C) for analysis.



#Example 1. Accessing List Items: Filtering Data

```
temperatures = [28, 31, 29, 32, 30, 33, 27]
```

```
threshold = 30
```

```
# Access temperatures above the threshold
```

```
high_temps = [temp for temp in temperatures if temp > threshold]
```

```
print("Temperatures above 30°C:", high_temps) # Output: [31, 32, 33]
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

COMMENTS

+ v ... ^ x

Output: [31, 32, 33]



2. Changing List Items: Normalizing Data

Problem: You have a list of product prices in dollars, and you need to convert them to euros using a conversion rate.



#Example 2. Changing List Items: Normalizing Data

```
prices_dollars = [100, 200, 300, 400]
```

```
conversion_rate = 270 # 1 USD = 270 PKR
```

```
# Change list items to convert prices to PKR
```

```
prices_pkr = [price * conversion_rate for price in prices_dollars]
```

```
print("Prices in PKR:", prices_pkr)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Microsoft Windows [Version 10.0.22631.5624]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\Dr Imtiaz Korejo\Desktop\pythonworkspace>
```



LEARNING RESOURCES

Source Reference used in Slide

- <https://www.w3schools.com/python>

Other Resources

- <https://realpython.com/>
- <https://docs.python.org/3/>
- <https://docs.python.org/3/tutorial/index.html>
- <https://www.tutorialspoint.com/python/index.htm>



Programming for AI

END of SESSION

Prof. Dr. Imtiaz Ali Korejo

Week- , Lecture-

